

admin.py – Pseudocode Design Document

Student Name: [Your Full Name] | Student Number: [Your Student Number]

Section 1: Main Program

Try to open locations.txt and read it, put the json data into a variable called data, then close the file

If that doesn't work, set data to an empty list

Print a welcome message

Start an infinite loop:

- Show the menu to the user (add, list, search, view, delete, map, quit options)

- Ask the user to type their choice, save it as lowercase in choice

- If choice equals "a":

 - Ask the user to type a city name using input_something, save it as city_name

 - Use the Geocoding API to search for city_name and get up to 4 results, save them in results

 - If results is empty:

 - Print "No matching locations."

 - If results has items:

 - Print how many locations were found

 - Go through each result, number them starting from 1, print the number and call format_location on each one, also print the full details

 - Call select_option with "Select a location (0 to cancel):" and the results list, save what comes back as selected

 - If selected is not False:

 - Make a new dictionary called new_location with name, country, lat rounded to 6 decimal places, lon rounded to 6 decimal places

 - If selected has a "state" key and it's not blank, add state to new_location

 - Add new_location to data

 - Call save_data with data

 - Print a confirmation message, include format_location of new_location

- If choice equals "l":

 - If data is empty, print "No locations saved."

 - Otherwise go through each location in data, number from 1, print the number and format_location of each one

- If choice equals "s":

 - If data is empty, print "No locations saved."

 - Otherwise:

 - Use input_something to ask for a search term, save it as search_term

 - Make a variable called found and set it to False

 - Loop through data with index starting at 0:

 - If search_term (lowercased) is somewhere inside format_location of that location (also lowercased):

 - Print index + 1 and format_location of that location

```

        Set found to True
    If found is still False, print "No results found."
If choice equals "v":
    If data is empty, print "No locations saved."
    Otherwise:
        Go through data, number from 1, print the number and format_location of each
        Call select_option with "Location number to view (0 to cancel):" and data, save as selected
        If selected is not False, print format_location of selected plus its lat and lon
If choice equals "d":
    If data is empty, print "No locations saved."
    Otherwise:
        Go through data, number from 1, print the number and format_location of each
        Call select_option with "Location number to delete (0 to cancel):" and data, save as selected
        If selected is not False:
            Remove selected from data
            Call save_data with data
            Print "Location deleted."
If choice equals "m":
    If data is empty, print "No locations saved."
    Otherwise:
        Go through data, number from 1, print the number and format_location of each
        Call select_option with "Location number to map (0 to cancel):" and data, save as selected
        If selected is not False:
            Build a Google Maps URL from selected's lat and lon
            Open the URL in a new browser tab
If choice equals "q":
    Print a goodbye message with name and student number
    Break out of the loop
If choice is anything else, print "Invalid choice."

```

Section 2: Functions

input_something(prompt)

Takes in a prompt string

Keeps looping until the user types something that isn't just spaces:

```
    Show the prompt, save what the user types as value
```

```
    If value isn't blank after stripping whitespace, return it
```

format_location(location)

Takes in a location dictionary

```
    If the dictionary has a "state" key, return name + state + country joined with ", "
```

```
    If there's no state, return name + country joined with ", "
```

select_option(prompt, options)

Takes in a prompt string and a list called options

Loops until the user picks something valid:

- Show the prompt, try to convert what the user types to a whole number, save as choice

- If choice is 0, return False

- If choice is between 1 and the length of options, return the item at choice minus 1

- If the user typed something that isn't a number, ignore it

- Print a message telling them to enter a valid number between 0 and the length of options

save_data(data)

Takes in the full data list

Opens locations.txt in write mode, writes data to it as formatted JSON, then closes the file

Doesn't return anything